

Ödev 09: Bir İfade Değerlendirici

Aritmetik ifadeleri mühürlü bir hiyerarşi olarak modelleyin, sonra derleyicinin bilerek unuttuğunuz bir durumu yakalamasına izin verin.

Bu, mühürlü tipler artı record'lar artı desen eşlemenin klasik kullanımındır ve bir kez elle yazmaya değer.

Birinci Bölüm: Model

`Calculator.java` dosyasını klasik biçimde oluşturun.

Beş record tipine izin veren bir `sealed interface Expr` tanımlayın:

Record	Bileşenler
<code>Literal</code>	bir <code>int value</code>
<code>Add</code>	iki <code>Expr</code>
<code>Multiply</code>	iki <code>Expr</code>
<code>Negate</code>	bir <code>Expr</code>
<code>Divide</code>	iki <code>Expr</code>

`Add`, `Multiply`, `Negate` ve `Divide` tiplerinin `int` değil `Expr` tuttuğuna dikkat edin. Yapıyı bir ağaç ve metotları özyinelemeli yapan şey budur.

İkinci Bölüm: Üç İşlem

Her biri `default` içermeyen tek bir `switch ifadesi` olan üç metot yazın:

- `int` sonucu döndüren `evaluate(Expr)`
- İkili işlemleri parantezleyerek okunaklı bir metin döndüren `print(Expr)`
- Ağacın kaç düğüm içerdiğini döndüren `countNodes(Expr)`

`(3 + 4) * -(2)` ifadesini kurun ve üçünü de üzerinde çalıştırın.

Bileşenleri çıkarmak için **record desenleri** kullanın. `case Add a` yazıp sonra `a.left()` çağırmak yerine `case Add(Expr left, Expr right)` yazın.

Üçüncü Bölüm: Derleyicinin İzlediğini Kanıtlayın

`permits` cümlesine altıncı bir record olan `Power` ekleyin. `evaluate` metodunu **güncellemeyin**.

Derleyin. Tam hatayı, isimlendirdiği eksik deseni dahil ederek bir yoruma kaydedin. Sonra ya `Power` durumunu ele alın ya da onu yeniden kaldırın.

`evaluate` metodunda bir `default` dalı olsaydı bunun yerine ne olacağını bir cümleyle yazın.

Dördüncü Bölüm: Sadeleştirme

Şu kurallar için **koşullar** kullanarak, daha basit ama eşdeğer bir `Expr` döndüren `simplify(Expr)` yazın:

Kural	Örnek
<code>0 + x</code> ve <code>x + 0</code>	<code>(0 + 7)</code> sonucu <code>7</code>
<code>1 * x</code> ve <code>x * 1</code>	<code>(1 * 9)</code> sonucu <code>9</code>
<code>0 * x</code> ve <code>x * 0</code>	<code>(0 * 9)</code> sonucu <code>0</code>
<code>--x</code>	<code>--5</code> sonucu <code>5</code>
sabit katlama	<code>(2 + 3)</code> sonucu <code>5</code>

Çift olumsuzlama kuralı ilginç olanı. **İç içe** bir record deseni istiyor, tek bir case etiketinde iki seviye eşleştiriyor.

Burada sıra önemli. Koşullu durumların aynı tip için koşulsuz olandan neden önce gelmesi gerektiğini bir yorumda söyleyin.

Kabul Kriterleri

- ☐ `java Calculator.java` ile çalışıyor
- ☐ `Expr` açık bir `permits` cümlesiyle `sealed`
- ☐ `evaluate`, `print` ve `countNodes` her biri `default` içermeyen tek switch
- ☐ Parçalamak için erişimci çağrılar değil record desenleri kullanılıyor
- ☐ Bir yorum ürettiğiniz gerçek "missing patterns" hatasını alınıyor
- ☐ Bir cümle `default` dalının neye mal olacağını açıklıyor
- ☐ `simplify` beş kuralın hepsini koşullarla uyguluyor
- ☐ Çift olumsuzlama kuralı iç içe bir record deseni kullanıyor
- ☐ Bir yorum koşullu-önce-koşulsuz sıralamasını açıklıyor

İleri Seviye

Sıfıra bölme, bir `int` döndürerek ele alınamaz. Sonucu, `Success(int value)` ve `Failure(String reason)` tiplerine izin veren ikinci bir mühürlü arayüz `Result` olarak modelleyin ve onu döndüren `evaluateSafely` yazın.

Sonra bunu istisna fırlatmakla karşılaştıran bir yorum yazın. Modül 10 sırada istisnaları anlatıyor, dolayısıyla her yaklaşımın çağırana neye mal olduğunu okumadan önce not edin.

İpucu, iç içe desen derlenmiyorsa

`Negate` bir `Expr` tutar, ve `Negate` kendisi de bir `Expr` tir. Dolayısıyla bileşeni başka bir `Negate` olan bir `Negate`, tipi iki kez isimlendirip ikisini de parçalayarak eşleştirilir. Dış deseni önce yazın ve içtekini parantezlerinin içine bırakın.